

IHK Übungsheft – BubbleSort 2017–2025

Dieses Heft enthält prüfungsnahe Aufgaben, Lösungen im IHK-Pseudocode (Java-ähnlich), Schritt-für-Schritt-Erklärungen, Beispiele und Theorie-Checks. Jeder Abschnitt passt auf eine Seite (beim Drucken), damit du dir leicht ein PDF-Heft erstellen kannst.

Kapitel 1 – Klassischer BubbleSort (Ganzzahlen, aufsteigend)

Aufgabe (IHK-Stil): Gegeben ist ein Feld `werte` mit ganzen Zahlen. Entwickeln Sie einen Algorithmus, der das Feld aufsteigend sortiert. Geben Sie den Pseudocode an.

Lösung (Pseudocode)

```
Funktion bubbleSort(werte: Array<Integer>)  
  für i = 0 bis länge(werte) - 2  
    für j = 0 bis länge(werte) - 2 - i  
      wenn werte[j] > werte[j + 1] dann  
        temp = werte[j]  
        werte[j] = werte[j + 1]  
        werte[j + 1] = temp  
      ende wenn  
    ende für  
  ende für  
Ende Funktion
```

Erklärung

- Äußere Schleife: $n-1$ Durchläufe (n = Länge).
- Innere Schleife: vergleicht Nachbarn und tauscht bei Bedarf.
- Nach jeder äußeren Runde ist das größte noch unsortierte Element am Ende.
- Laufzeit: **$O(n^2)$** ; in-place; stabil.

Beispiel

Schritt	Array
Start	[5, 2, 9, 1]
Runde 1	[2, 5, 1, 9]
Runde 2	[2, 1, 5, 9]
Runde 3	[1, 2, 5, 9]

Komplexität: $O(n^2)$

Speicher: $O(1)$

stabil

Kapitel 2 – BubbleSort für Strings (alphabetisch)

Aufgabe: Sortieren Sie ein `Array<String>` alphabetisch aufsteigend.

Pseudocode

```
Funktion bubbleSortStrings(namen: Array<String>)
  für i = 0 bis länge(namen) - 2
    für j = 0 bis länge(namen) - 2 - i
      wenn namen[j] > namen[j + 1] dann    // lexikographischer Ve
        t = namen[j]
        namen[j] = namen[j + 1]
        namen[j + 1] = t
      ende wenn
    ende für
  ende für
Ende Funktion
```

Hinweis: Im IHK-Pseudocode genügt der Vergleich `>` für Strings (lexikographische Ordnung).

Mini-Check

- „Anna“, „Anja“, „Bert“, „Zoo“ ? bereits sortiert?
 - Wie verhält sich der Algorithmus bei Groß/Kleinschreibung? (Prüfung: oft egal, sonst vorher vereinheitlichen.)
-

Kapitel 3 – Optimierter BubbleSort (mit Abbruch)

Aufgabe: Ergänzen Sie BubbleSort um eine Abbruchbedingung, wenn in einer Runde kein Tausch stattfand.

Pseudocode

```
Funktion bubbleSortOptimiert(werte: Array<Integer>)  
    getauscht = true  
    solange getauscht  
        getauscht = false  
        für i = 0 bis länge(werte) - 2  
            wenn werte[i] > werte[i + 1] dann  
                t = werte[i]  
                werte[i] = werte[i + 1]  
                werte[i + 1] = t  
                getauscht = true  
            ende wenn  
        ende für  
    ende solange  
Ende Funktion
```

Erklärung

Wenn in einer kompletten Runde kein Tausch mehr vorkommt, ist das Feld bereits sortiert ?
früher Abbruch. Im Bestfall **$O(n)$** , im Worst-Case weiterhin **$O(n^2)$** .

Kapitel 4 – BubbleSort mit Comparator (z. B. Tageskurse)

Aufgabe: Sortieren Sie `Tageskurs[]` mithilfe einer Vergleichsfunktion `vergleiche(a, b)` (liefert <0 , $=0$, >0).

Pseudocode

```
Funktion sort(kurse: Array<Tageskurs>, vergleiche: Funktion) : void
    für i = 0 bis länge(kurse) - 2
        für j = 0 bis länge(kurse) - 2 - i
            wenn vergleiche(kurse[j], kurse[j+1]) > 0 dann
                t = kurse[j]
                kurse[j] = kurse[j+1]
                kurse[j+1] = t
            ende wenn
        ende für
    ende für
Ende Funktion
```

So wurde in AP2 (Sommer 2022, Aufgabe 2.2) die Sortieridee oft abgefragt: nicht „Bubble“ nennen, aber Vergleichslogik korrekt einsetzen.

Kapitel 5 – Fehleranalyse (typische IHK-Aufgabe)

Aufgabe: Im folgenden Pseudocode fehlt/fehlt etwas. Markieren und korrigieren Sie den Fehler.

```
// fehlerhaft:
Funktion bubbleSort(a: Array<Integer>)
    für i = 0 bis länge(a) - 1      // Fehler: -1 führt zu Out-of-Bounds
        für j = 0 bis länge(a) - 1 - i
            wenn a[j] > a[j+1] dann
                t = a[j]; a[j] = a[j+1]; a[j+1] = t
            ende wenn
        ende für
    ende für
Ende Funktion
```

Korrektur

```
für i = 0 bis länge(a) - 2
    für j = 0 bis länge(a) - 2 - i
```

Merksatz: Äußere Schleife bis $n-2$, innere bis $n-2-i$.

Kapitel 6 – Vergleich BubbleSort vs. InsertionSort (Theorie)

- **BubbleSort:** sehr einfach, stabil, in-place, aber $O(n^2)$; gut zum Lehren/Prüfungen.
- **InsertionSort:** ebenfalls $O(n^2)$, aber schneller auf (fast) sortierten Daten; stabil; in-place.
- **Wann welcher?** Kleine Datenmengen, einfache Implementierung ? beide okay; echte Anwendungen nutzen meist effizientere Verfahren.

InsertionSort (zum Vergleich)

```
Funktion insertionSort(a: Array<Integer>)  
  für i = 1 bis länge(a) - 1  
    key = a[i]  
    j = i - 1  
    solange j >= 0 und a[j] > key  
      a[j+1] = a[j]  
      j = j - 1  
    ende solange  
    a[j+1] = key  
  ende für  
Ende Funktion
```

Kapitel 7 – BubbleSort Schritt-Simulation (Prüfungstechnik)

Aufgabe: Simulieren Sie die erste und zweite äußere Runde für [7, 3, 4, 2].

Lösungsskizze

Vergleich	Ergebnis
(7,3)	[3, 7, 4, 2]
(7,4)	[3, 4, 7, 2]
(7,2)	[3, 4, 2, 7] // Ende Runde 1: 7 „blubbert“ ans Ende
(3,4)	[3, 4, 2, 7]
(4,2)	[3, 2, 4, 7] // Ende Runde 2

Tipp: Markiere nach jeder Runde das „fixierte“ Ende.

Kapitel 8 – Theorie-Check (MC-ähnlich)

1. Die Worst-Case-Zeit von BubbleSort ist ...

- ? $O(1)$? $O(n)$? $O(n^2)$? $O(n \log n)$

2. BubbleSort ist ...

- ? stabil ? nicht stabil ? benötigt $O(n)$ Zusatzspeicher

3. Ein sinnvoller Abbruch ist möglich, wenn ...

- ? in einer Runde kein Tausch stattfand ? das erste Element korrekt ist

4. Nach jeder äußeren Runde liegt am Ende ...

- ? das kleinste Element ? das größte Element des unsortierten Bereichs
-

Kapitel 9 – Zusammenfassung & Spickzettel

- **Pseudocode (Kern):** zwei Schleifen; Nachbarn tauschen; inneres Ende $n-2-i$.
- **Optimierung:** Flag `getauscht` ? früher Abbruch; Best Case $O(n)$.
- **Eigenschaften:** stabil, in-place, einfach; aber langsam für große n .
- **Prüfungstaktik:** erst Schleifen-Schablone schreiben, dann Vergleich + Tausch. Bei Fehleranalyse auf Grenzen ($n-2$) achten.

```
Funktion bubbleSort(werte: Array<Integer>)  
  für i = 0 bis länge(werte) - 2  
    für j = 0 bis länge(werte) - 2 - i  
      wenn werte[j] > werte[j + 1] dann  
        t = werte[j]; werte[j] = werte[j+1]; werte[j+1] = t  
      ende wenn  
    ende für  
  ende für  
Ende Funktion
```

Merke: Namen, Idee, Pseudocode und O-Notation sicher beherrschen. Das reicht, um BubbleSort-Aufgaben zuverlässig zu lösen.